Python Implementation:
California Housing Price Prediction

NISHANT

June 17, 2026

---

1. Importing Necessary Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

2. Loading the Dataset

```
# Loading the dataset
print("Loading California Housing Dataset...")
data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename('MedHouseVal')], axis=1)

# Checking for missing values
print("\n--- Missing Values Check ---")
print(df.isnull().sum())
```

Output:

```
Loading California Housing Dataset...

--- Missing Values Check ---
MedInc         0
HouseAge       0
AveRooms       0
AveBedrms      0
Population     0
AveOccup       0
Latitude       0
Longitude      0
MedHouseVal    0
dtype: int64
```

3. Exploratory Data Analysis (EDA)

```
# Plotting Correlation Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix of California Housing Features')
plt.show()
```

4. Data Splitting and Model Training

```
# X contains features, y contains the target variable
X = df.drop(columns='MedHouseVal')
y = df['MedHouseVal']

# Splitting data: 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Training Linear Regression Model
print("Training Linear Regression Model...")
model = LinearRegression()
model.fit(X_train, y_train)

# Making predictions on the test set
y_pred = model.predict(X_test)
print("Model Training Complete!")
```

Output:

```
Training Linear Regression Model...
Model Training Complete!
```

5. Model Evaluation and Results

```
# Calculating metrics
mae  = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2   = r2_score(y_test, y_pred)

print(f"MAE: {mae:.3f} | RMSE: {rmse:.3f} | R2: {r2:.3f}")

# Plotting Actual vs Predicted
plt.scatter(y_test, y_pred, alpha=0.4)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```

Output:

```
MAE: 0.533 | RMSE: 0.746 | R2: 0.576
```